

Relatório Lab 2 - ENG4448

Vivi Arruda Pereira - 2110624

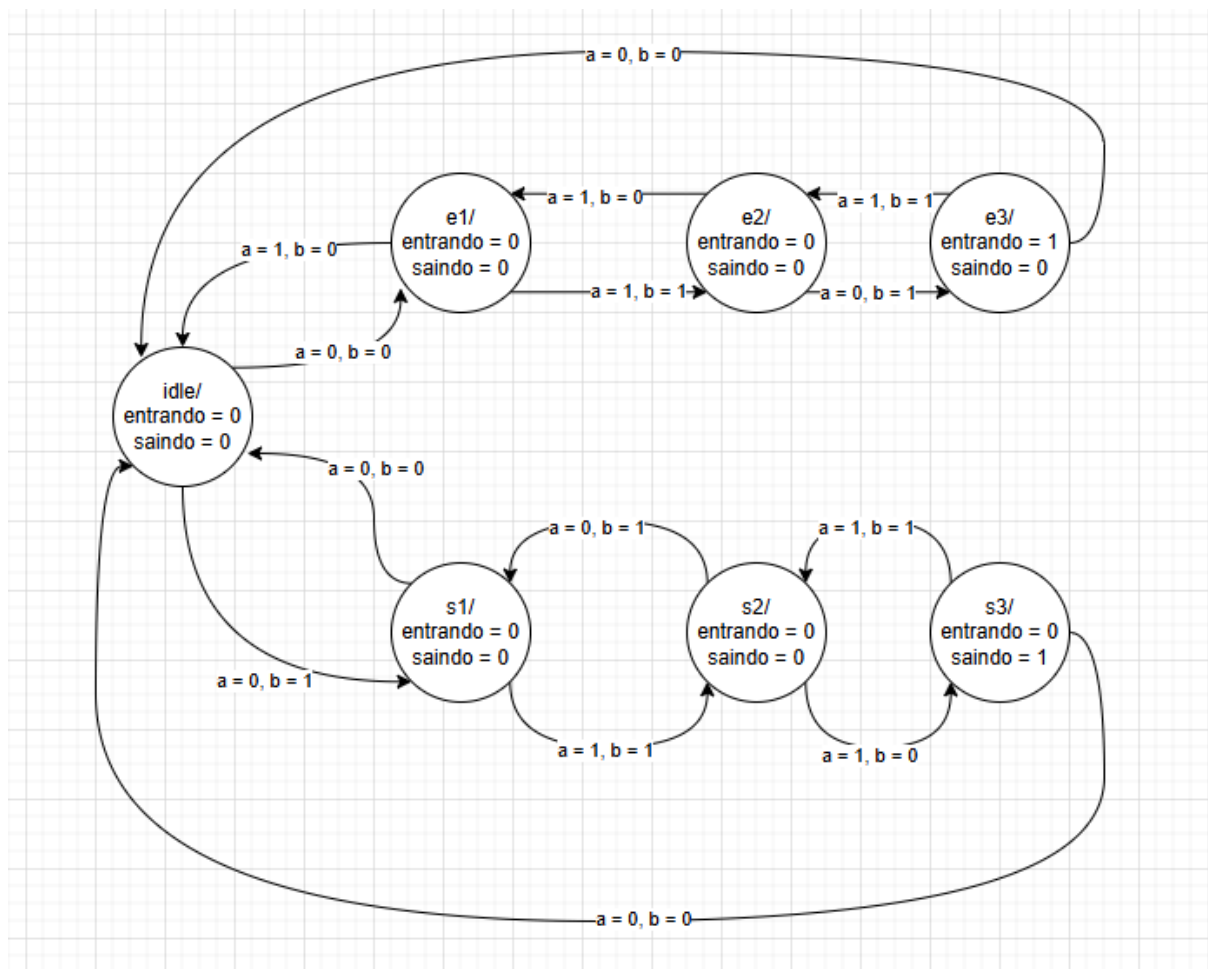
Vinícius Machado da Rocha Viana - 2111343

1) Como o teste prático na FPGA utiliza botões mecânicos (push-buttons) para simular os sensores infravermelhos da cancela, lidamos com um problema físico: Quando pressionamos ou soltamos um botão, os contatos metálicos internos vibram por alguns milissegundos antes de se estabilizarem completamente.

Como a placa opera em uma alta frequência de clock, ela processa eventos de forma muito rápida. Sem o debounce, a FPGA leria essa vibração metálica como se o carro estivesse entrando e saindo dezenas de vezes em uma fração de segundo.

Esse circuito funciona como um filtro: ele aguarda o sinal elétrico ficar constante por um período de tempo predeterminado antes de repassar a leitura de "1" ou "0" para o resto da lógica, garantindo que o acionamento seja limpo e confiável.

2.1)



A máquina de estados proposta é do tipo **Moore**. Isso porque a saída do sistema (a atualização da contagem de vagas) depende apenas do estado atual em que a máquina está, e não diretamente do valor das entradas (os sensores A e B).

Escolhemos esse modelo porque a contagem de carros só deve mudar quando a máquina alcança o estado final do ciclo de entrada ou de saída. Se fosse uma máquina de Mealy, a saída mudaria na mesma hora em que os sensores fossem acionados, o que poderia causar erros na contagem se uma pessoa passasse pela cancela ou desse algum ruído no fio

Significado de cada estado

- **Idle:** É o estado de repouso. A cancela está vazia e nenhum sensor está bloqueado (sensores $A=0$ e $B=0$). O sistema aguarda alguma ação.

Fluxo de Entrada (E1 a E3):

- **E1:** Um carro começa a entrar, interrompendo apenas o primeiro feixe infravermelho ($A=1$, $B=0$).
- **E2:** O carro avança e interrompe ambos os feixes simultaneamente ($A=1$, $B=1$).
- **E3:** O carro continua avançando, liberando o primeiro sensor, mas mantendo a traseira no segundo ($A=0$, $B=1$).
- Após o estado E3, quando ambos os sensores vão para 0, a máquina retorna ao estado **Idle** e a saída envia o comando para **incrementar** a contagem de carros.

Fluxo de Saída (S1 a S3):

Ocorre exatamente a mesma lógica, mas no sentido inverso, mapeando a trajetória do carro saindo do estacionamento.

- **S1:** O carro bloqueia o sensor de dentro ($A=0$, $B=1$).
- **S2:** O carro bloqueia ambos ($A=1$, $B=1$).
- **S3:** O carro bloqueia apenas o sensor de fora ($A=1$, $B=0$).
- Ao liberar o último sensor, a máquina volta ao **Idle** e a saída **decrementa** a contagem.

Se em qualquer momento (como no E1 ou S1) a sequência for quebrada e os sensores voltarem a zero sem passar pelo estado do meio (E2/S2), o carro "desistiu" ou era apenas uma pessoa passando. Nesse caso, a máquina volta direto para o **Idle** sem alterar a contagem de carros.

2.2)

a)

Device Utilization Summary (estimated values)			
Logic Utilization	Used	Available	Utilization
Number of Slices	4	4656	0%
Number of Slice Flip Flops	3	9312	0%
Number of 4 input LUTs	8	9312	0%
Number of bonded IOBs	12	232	5%
Number of GCLKs	1	24	4%

b)

Timing Summary:

Speed Grade: -4

Minimum period: 3.123ns (Maximum Frequency: 320.205MHz)

Minimum input arrival time before clock: 4.272ns

Maximum output required time after clock: 6.067ns

Maximum combinational path delay: 6.834ns

Timing Detail:

All values displayed in nanoseconds (ns)

=====

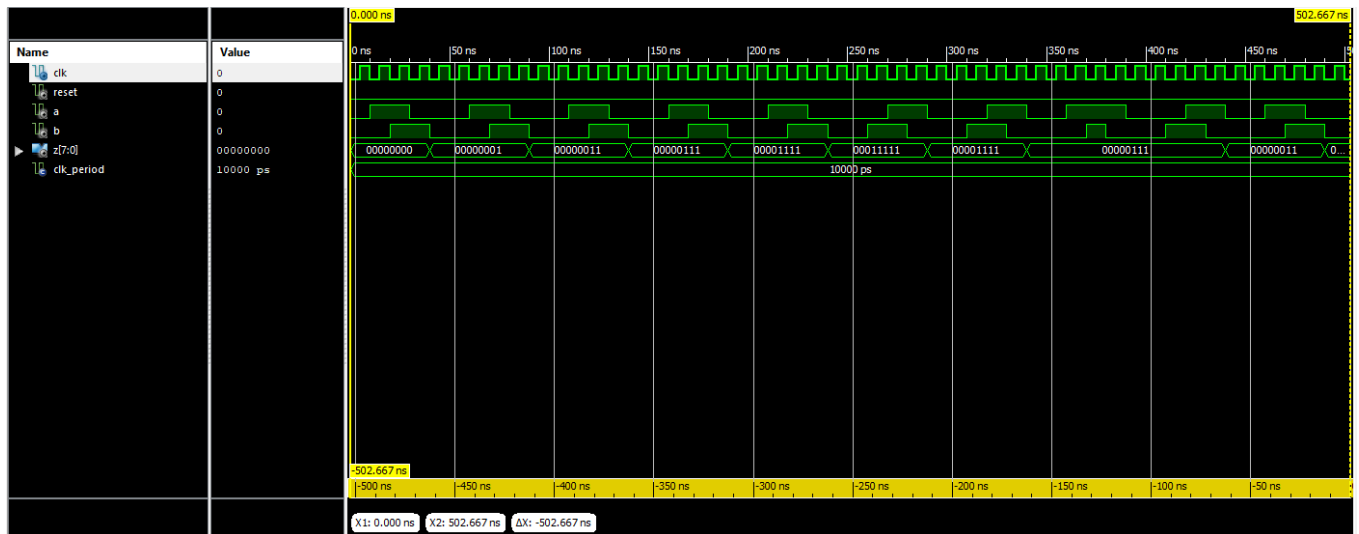
Timing constraint: Default period analysis for Clock 'clk'

Clock period: 3.123ns (frequency: 320.205MHz)

Total number of paths / destination ports: 13 / 3

Delay: 3.123ns (Levels of Logic = 2)

2.3)



O resultado após as etapas indica quantos leds foram acesos, seguindo o teste que foi pedido. a saída “z” começa com todos os bits zerados, eventualmente vai sendo realizado 1 bit shift para a esquerda toda vez que um carro entra e um para a direita toda vez que um carro sai. Esse comportamento é observado à medida que o “00000000” se torna “00000001” e depois “00000011” e assim por diante

2.4) [Link do Vídeo](#)